

จากโค้ดสู่ PROEN Cloud, GitOps & Argo CD เวอร์กช็อป

การออกแบบสถาปัตยกรรม, คอนเทนเนอร์ไรเซชัน, ความปลอดภัยห่วงโซ่อุปทาน และการส่งมอบไมโครเซอร์วิสอย่าง
ต่อเนื่องบน Cloud-Native

องค์กรผู้จัดงาน

สมาคมโปรแกรมเมอร์ไทย

Thai Programmer Association

พันธมิตรภาคอุตสาหกรรม

PROEN Corp & VMware by Broadcom

Cloud IDC & Modern App Platform

วันและสถานที่จัดงาน

วันจันทร์ที่ 21 กันยายน 2569

ClassRoom 3, True Digital Park,
กรุงเทพฯ

GITHUB คลังโค้ดลับ

3IVv0w/code-to-cloud-workshop

คู่มือการเรียนรู้และสคริปต์เวิร์กช็อปอย่างเป็นทางการ

ตารางเวลาหลักสูตรและขั้นตอนการปฏิบัติการ

เส้นทางการเรียนรู้เต็มวันอย่างมีแบบแผน ตั้งแต่การออกแบบสถาปัตยกรรมซอร์สโค้ดจนถึงการกระทบยอดคลัสเตอร์อัตโนมัติ

ช่วงเช้า: พื้นฐานและความปลอดภัย (08:30 – 12:00)

08:30 – 09:00 **Mod 00:** ลงทะเบียน ปฐมนิเทศ และตรวจเช็คความพร้อมเครื่องเวิร์กสเตชัน

09:00 – 09:30 **Mod 01:** เส้นทาง Code to Cloud และหลักการ 12-Factor App ยุคใหม่

09:30 – 10:30 **Mod 02:** กรอบความคิดการเรียนรู้และวัฒนธรรมวิศวกรรมประสิทธิภาพสูง

10:30 – 10:45 พักรับประทานอาหารว่างช่วงเช้า (15 นาที)

10:45 – 11:15 **Mod 03:** สถาปัตยกรรมไมโครเซอร์วิสและการรองรับกราฟฟิคมหาศาล

11:15 – 12:00 **Mod 04:** ความปลอดภัยห่วงโซ่อุปทานซอฟต์แวร์และ DevSecOps

12:00 – 13:00 พักรับประทานอาหารกลางวันและสร้างเครือข่ายความร่วมมือ (60 นาที)

ช่วงบ่าย: แพลตฟอร์มและการส่งมอบ (13:00 – 17:00)

13:00 – 13:20 **Mod 05:** บรรยายพิเศษ VMware by Broadcom: "นี่คือซอร์สโค้ด รันบนคลาวด์ได้เลย"

13:20 – 14:00 **Mod 06:** บริการคลาวด์, กลยุทธ์การดีพลอย และ Full-Stack Observability

14:00 – 15:00 **Mod 07:** ระบบควบคุมเวอร์ชัน Git วนิยายการพัฒนาและมาตรฐานระดับองค์กร

15:00 – 15:30 พักรับประทานอาหารว่างช่วงบ่าย (30 นาที)

15:30 – 16:00 **Mod 08:** หลักการ GitOps หัวใจสำคัญและความมั่นคงปลอดภัยแบบ Pull-Based

16:00 – 16:45 **Mod 09:** เวิร์กช็อปปฏิบัติการจริง: ส่งมอบและบริหารจัดการแอปด้วย Argo CD

16:45 – 17:00 **Mod 10:** สรุปบทเรียน, การดูแลระบบ Day-2, แผนงานองค์กร และช่วงถาม-ตอบ

พิมพ์เขียวการส่งมอบระบบ Cloud-Native แบบครบวงจร

การแยกขอบเขตความรับผิดชอบอย่างเคร่งครัด ระหว่างพื้นที่ทำงานนักพัฒนา, ห่วงโซ่อุปทาน และเอนจิน GitOps ภายในคลัสเตอร์

1. พื้นที่ทำงานนักพัฒนา (Developer Workspace)

Trunk-Based Git Push · Conventional Commits · ลายมือชื่อ SSH เข้ารหัส

↓ **ทริกเกอร์ Webhook / CI อัตโนมัติ**

2. ไปป์ไลน์ห่วงโซ่อุปทานและการสร้างระบบ (CI Pipeline)

Hermetic Build · CycloneDX SBOM · ตรวจสอบ CVE ด้วย Trivy · ลงลายมือชื่อด้วย Cosign

↓ **push Image Digest สู่ Registry + ยื่น PR สู่ Config Repo**

3. เอนจินระบยยอดสถานะ: GitOps (Argo CD)

Pull-Based จากภายในคลัสเตอร์ · จัดลำดับด้วย Sync Waves · กู้คืนระบบอัตโนมัติ (Self-Healing)

↓ **บังคับใช้สถานะเป้าหมายบนคลัสเตอร์จริง**

4. คลัสเตอร์ Kubernetes บน PROEN Cloud

Distroless Pods ไร้ Shell · PostgreSQL Row Locks ป้องกันขายตั๋วเกิน · OpenTelemetry

1. ปลอด Credential คลัสเตอร์ใน CI ภายนอก

GitHub Actions หรือ GitLab runners ภายนอกไม่มีกุญแจ cluster-admin ใดๆ คลัสเตอร์ปิดพอร์ตตาเข้าทั้งหมด ป้องกันการรั่วไหลแบบเบ็ดเสร็จ

2. หลักฐานความถูกต้องของแหล่งกำเนิด (Provenance)

คอนเทนเนอร์อิมเมจทุกตัวต้องผ่านการลงลายมือชื่อดิจิทัลผ่าน Cosign และแนบ SBOM หากไม่ผ่านการเซ็นจะไม่สามารถนำขึ้นรันบนคลัสเตอร์ได้

3. ตรวจสอบการเบี่ยงเบนและกู้คืนอัตโนมัติ (Self-Healing)

การแก้ไขสดบนคลัสเตอร์ (`kubectl edit`) จะถูกตรวจจับเป็น `OutOfSync` ทันที และระบบจะย้อนค่ากลับสู่สิ่งที่ระบุใน Git ภายในไม่กี่วินาที

ปัญหาคลาสสิก: 'มันรันบนเครื่องผมได้นะ'

ทำไมความหลากหลายของเครื่องเวิร์กสเตชันถึงสร้างความล่าช้าและหายนะในการอบรมคลาวด์

ฝันร้ายในวันเริ่มต้นของงานวิศวกรรม

ในการจัดอบรมและขั้นตอนการ Onboard วิศวกรระดับองค์กร **เวลาเฉลี่ย 40% มักสูญเปล่า** กับการแก้ไขปัญหาเฉพาะเครื่อง เช่น PATH ไม่ตรงกัน, รุ่นของ CLI ไม่เข้ากัน, สิทธิ์ Docker socket ขัดข้อง หรือ Helm charts ตกຸ່ນ

เมื่อวิศวกรเอ่ยว่า *"มันรันบนเครื่องผมได้นะ"* นั้นสะท้อนถึงสภาพแวดล้อมที่ควบคุมไม่ได้และไม่สามารถสร้างซ้ำได้ในระดับโปรดักชัน

จุดผิดพลาดที่พบบ่อยบนเครื่องเวิร์กสเตชัน

- เวอร์ชันของ `kubectl` ฝั่ง Client ไม่ตรงกับ API Server ของคลัสเตอร์
- ปัญหาสิทธิ์ของ Docker Desktop หรือการล็อกของ Credential Socket
- Git รุ่นต่ำกว่า 2.34 ที่ไม่รองรับการลงลายมือชื่อคอมมิตด้วยกุญแจ SSH
- การขาด OIDC Token Support สำหรับการรับรองอิมเมจผ่าน Cosign

เป้าหมายมาตรฐาน: วันนี้เราจะบังคับใช้การตรวจสอบ Pre-Flight แบบอัตโนมัติอย่างเข้มงวด เพื่อรับรองว่าเครื่องของผู้เข้าร่วมทุกคนพร้อมใช้งานด้วยชุดเครื่องมือที่แม่นยำและเสถียร 100%

ปฏิบัติการตรวจสอบความพร้อมของเครื่องเวิร์กสเตชัน

รันสคริปต์ตรวจสอบสภาพแวดล้อมอัตโนมัติ เพื่อยืนยันความถูกต้องของเครื่องมือ CLI ประจำตัว

คำสั่งในเทอร์มินัล (รองรับทุกระบบปฏิบัติการ)

```
# 1. โคลนคลังโค้ดเวิร์กช็อปอย่างเป็นทางการ
git clone https://github.com/3lVv0w/code-to-cloud-workshop.git
cd code-to-cloud-workshop/workshop-setup

# 2. รันสคริปต์ตรวจสอบ Pre-Flight Audit:
# สำหรับ macOS & Linux (Terminal):
chmod +x 01-prerequisites/check-env.sh
./01-prerequisites/check-env.sh

# สำหรับ Windows (PowerShell):
.\01-prerequisites\check-env.ps1

# 3. ผลลัพธ์ที่ถูกต้องบนหน้าจอคอนโซล:
[FOUND] Git Version Control: git version 2.43.0
[FOUND] Docker Engine / CLI: Docker version 26.1.1
[FOUND] Kubernetes CLI: Client Version: v1.30.2
[FOUND] Helm 3 Package Manager: v3.14.4
[FOUND] Argo CD CLI: argocd: v2.11.0
[FOUND] Cosign (Sigstore): cosign version 2.2.4
SUCCESS: All required CLI tools are present!
```

คู่มือการติดตั้งเครื่องมือข้ามแพลตฟอร์ม

macOS (ผ่าน Homebrew):

```
brew install git docker kubectl helm argocd cosign
```

Windows (ผ่าน Winget ใน PowerShell):

```
winget install Git.Git Docker.DockerDesktop Kubernetes.kubectl Helm.Helm Sigstore.Cosign
```

หมายเหตุ: หากเครื่องยังขาดเครื่องมือเสริมใดๆ ตัวสคริปต์จะพิมพ์คำสั่งติดตั้งที่ตรงกับระบบปฏิบัติการของคุณให้อัตโนมัติทันที

รายการตรวจสอบความพร้อมและเกณฑ์มาตรฐานก่อนเริ่มเวิร์กช็อป

ยืนยันความพร้อมของเครื่องมือในระดับ Baseline ก่อนก้าวสู่สถาปัตยกรรมระดับสูง

Git ≥ 2.34

การควบคุมเวอร์ชัน

จำเป็นต้องใช้สำหรับเซ็นรับรอง SSH commit signatures ในโมดูล 07

Docker / OCI

คอนเทนเนอร์รันไทม์

จำเป็นสำหรับทดสอบ Distroleless, Buildpacks และ Docker Compose ในเครื่อง

Kubectl ≥ 1.28

การสื่อสารคลัสเตอร์

ใช้เชื่อมต่อไปยังคลัสเตอร์ Kubernetes ของ PROEN Cloud อย่างปลอดภัย

Argo CD CLI

การควบคุม GITOPS

ใช้ในการล็อกอิน ตรวจสอบสถานะ Application และสั่ง sync แอประดับโปรดักชัน

สถานะความพร้อม: เมื่อรันสคริปต์ผ่านแล้ว เวิร์กสเตชันของคุณจะอยู่ในสถานะ Deterministic และพร้อมสำหรับการทดลองจริงในทุกบทเรียน

ความเปราะบางของเซิร์ฟเวอร์แบบ 'สัตว์เลี้ยง' (Pets)

ทำไมแอปพลิเคชันแบบดั้งเดิมที่ยึดติดกับสถานะเครื่อง จึงล้มเหลวในการสเกลบนระบบคลาวด์

วิธีคิดแบบเก่า: ดูแลเซิร์ฟเวอร์ดังสัตว์เลี้ยง (Pets)

- ไฟล์คอนฟิกเฉพาะที่: บันทึกค่าเชื่อมต่อฐานข้อมูลลงใน `appsettings.json` หรือไฟล์ `.ini` บนฮาร์ดดิสก์
- เก็บไฟล์ที่ผู้ใช้อัปโหลดไว้ในเครื่อง: เซฟรูปลงใน `/var/data/uploads` หากเครื่องดับ ข้อมูลผู้ใช้จะสูญหายทันที
- การเชื่อมต่อเหนียวแน่น (Sticky Sessions): เก็บเซสชันการล็อกอินไว้ในแรมของเครื่องเดียว ส่งผลให้กระจายโหลดไม่ได้
- ปิดระบบแบบตัดไฟทันที: ปิดเครื่องกะทันหันทำให้คำสั่งตัดเงินของลูกค้าค้างอยู่กลางทาง

กระบวนทัศน์ Cloud-Native: ฝูงปศุสัตว์ (Cattle)

บนระบบ Kubernetes เซิร์ฟเวอร์และ Pod เป็นเพียง **สิ่งชั่วคราวพร้อมทดแทนได้ตลอดเวลา (Ephemeral Resources)**

- Pod สามารถถูกไล่ที่ (Evict) หรือปิดตัวได้ทุกเมื่อโดยไม่มีการเตือนล่วงหน้า
- แอปพลิเคชันต้องสามารถบูตขึ้นมาพร้อมรับงานได้ภายในเวลาไม่กี่วินาที
- สถานะทุกอย่างต้องฝากไว้ใน Backing Services ที่แยกต่างหาก (Redis, S3, PostgreSQL)

สัญญา 12-Factor สำหรับ Kubernetes: ปัจจัยที่ 1 – 6

รากฐานสถาปัตยกรรมสำหรับแอปพลิเคชันยุคใหม่ เพื่อความยืดหยุ่นและการทำงานแบบไร้สถานะบนคลาวด์

ข้อกำหนดทางการ: <https://12factor.net/> ↗ (Adam Wiggins, Heroku)

ปรับปรุงสำหรับ Cloud-Native Kubernetes & OCI Containers

FACTOR I · K8S CRITICAL

12factor.net/codebase ↗

I. ฐานโค้ด (Codebase)

หนึ่งฐานโค้ดถูกติดตามด้วยระบบควบคุมเวอร์ชัน (Git) หนึ่งชุด นำไปดีพลอยได้หลายสภาพแวดล้อมอย่างอิสระ

ประยุกต์บน K8s: บิลด์เป็น OCI Container Image Digest เดียว แล้วให้ Argo CD อ้างอิง Digest นี้กระจายสู่ทุกคลัสเตอร์

FACTOR II · K8S CRITICAL

12factor.net/dependencies ↗

II. แพ็กเกจที่ต้องพึ่งพา (Dependencies)

ประกาศและแยกส่วนไลบรารีภายนอกอย่างชัดเจนผ่าน Manifest ห้ามพึ่งพาโปรแกรมที่ติดตั้งไว้ล่วงหน้าบนโฮสต์

ประยุกต์บน K8s: ใช้ Dockerfile หรือ Buildpacks แพ็กเกจไบนารีและ Dependencies เข้าสู่ Distroless Container

FACTOR III · K8S CRITICAL

12factor.net/config ↗

III. การตั้งค่าระบบ (Config)

แยกการตั้งค่าออกจากซอร์สโค้ดอย่างเด็ดขาด เก็บค่าคอนฟิกไว้ในตัวแปรสภาพแวดล้อม (Environment Variables)

ประยุกต์บน K8s: ป้อนค่าผ่าน Kubernetes ConfigMap และ Secret หรือใช้ External Secrets Operator ดึงจาก Vault

FACTOR IV · K8S CRITICAL

12factor.net/backing-services ↗

IV. บริการสนับสนุน (Backing Services)

มองบริการที่เชื่อมต่อ (Postgres, Redis, RabbitMQ) เสมือนทรัพยากรภายนอกที่แนบมาผ่าน Network URI

ประยุกต์บน K8s: อ้างอิงผ่าน K8s Service DNS เช่น postgres.production.svc.cluster.local สลับเป็น DBaaS ได้โดยไม่ต้องแก้ไขโค้ด

FACTOR V · K8S CRITICAL

12factor.net/build-release-run ↗

V. สร้าง, ออกรุ่น, ปฏิบัติการ (Build, Release, Run)

แยกขั้นตอนการบิลด์โค้ด, การผูกคอนฟิกเข้ากับอิมเมจ (Release) และการรันจริง (Run) ออกจากกันอย่างเด็ดขาด

ประยุกต์บน K8s: CI สร้างอิมเมจ sha256 แบบไม่เปลี่ยนแปลง; GitOps บันทึก Release; K8s สั่งรัน Pod ห้ามแก้ไขโค้ดสดบนรันไทม์

FACTOR VI · K8S CRITICAL

12factor.net/processes ↗

VI. โพรเซสการทำงาน (Processes)

รันแอปพลิเคชันเป็นหนึ่งหรือหลายโพรเซสที่ไร้สถานะ (Stateless) ข้อมูลถาวรต้องส่งไปเก็บใน Datastore ภายนอกเท่านั้น

ประยุกต์บน K8s: รันเป็น Stateless Pods ช่วยให้ K8s ย้ายเครื่อง, สเกลเพิ่มลด, และทำ Rolling Updates ได้อย่างไร้รอยต่อ

สัณญา 12-Factor สำหรับ Kubernetes: ปัจจัยที่ 7 – 12

ความสามารถในการขยายขนาด, วงจรชีวิตการทำงานอย่างนุ่มนวล และมาตรฐาน Observability ในระบบแบบกระจาย

ข้อกำหนดทางการ: <https://12factor.net/> ↗ (Adam Wiggins, Heroku)

ปรับปรุงสำหรับ Cloud-Native Kubernetes & OCI Containers

FACTOR VII · K8S CRITICAL12factor.net/port-binding ↗

VII. การผูกพอร์ตเชื่อมต่อ (Port Binding)

ให้บริการ HTTP/gRPC ผ่านการผูกพอร์ตในตัวเอง ไม่พึ่งพาการถูกติดตั้งลงในเว็บเซิร์ฟเวอร์ภายนอก (เช่น Apache หรือ IIS)

ประยุกต์บน K8s: แอปเปิดพอร์ต :3000 เชื่อมผ่าน containerPort และ ClusterIP แล้วเราท์ผ่าน Ingress

FACTOR VIII · K8S CRITICAL12factor.net/concurrency ↗

VIII. การทำงานพร้อมกัน (Concurrency)

สเกลขยายขนาดในแนวนอนผ่าน Process Model แทนการล็อก Thread ขนาดใหญ่บนเครื่องเซิร์ฟเวอร์เครื่องเดียว

ประยุกต์บน K8s: จัดการผ่าน spec.replicas และปรับขยายอัตโนมัติด้วย HorizontalPodAutoscaler (HPA)

FACTOR IX · K8S CRITICAL12factor.net/disposability ↗

IX. ความพร้อมสับเปลี่ยน (Disposability)

บูตระบบได้รวดเร็ว (< 5 วินาที) และหยุดการทำงานอย่างนุ่มนวลผ่านสัญญาณ SIGTERM เพื่อเคลียร์คำขอที่ค้างอยู่

ประยุกต์บน K8s: ดักจับ SIGTERM , ปลดตัวออกจาก readinessProbe , และเคลียร์ Connection Pool ให้หมดจด

FACTOR X · K8S CRITICAL12factor.net/dev-prod-parity ↗

X. ความเหมือนกันของ Dev/Prod (Parity)

รักษาสภาพแวดล้อมของ Development, Staging และ Production ให้ใกล้เคียงกันที่สุด ใช้เครื่องมือและเวอร์ชันเดียวกัน

ประยุกต์บน K8s: ใช้อิมเมจ Digest เดียวกันในทุกสภาพแวดล้อม และปรับเปลี่ยนค่าเฉพาะจุดด้วย Kustomize Overlays

FACTOR XI · K8S CRITICAL12factor.net/logs ↗

XI. ข้อมูลบันทึกเหตุการณ์ (Logs)

มองล็อกเสมือนกระแสข้อมูลเหตุการณ์ (Event Streams) ห้ามเขียนลงไฟล์ในเครื่อง ให้ผ่าน JSON ออกทาง stdout โดยตรง

ประยุกต์บน K8s: Containerd รวบรวมสตรีม stdout/stderr แล้วให้ DaemonSets (Promtail/FluentBit) ส่งต่อไปยัง Grafana Loki

FACTOR XII · K8S CRITICAL12factor.net/admin-processes ↗

XII. โพรเซสการดูแลระบบ (Admin Processes)

รันงานดูแลระบบ เช่น การทำ Database Migration หรือสคริปต์ซ่อมแซม ในสภาพแวดล้อมเดียวกันกับแอปหลักแบบ One-Off

ประยุกต์บน K8s: รันเป็น Kubernetes Job หรือจัดลำดับใน Argo CD Sync Wave 0 ให้รันเสร็จสมบูรณ์ก่อนปล่อยแอปใหม่

ปฏิบัติการจัดการ Graceful Shutdown และ K8s Health Probes

ตรวจสอบและทดสอบไมโครเซอร์วิสตัวอย่าง 12-Factor พร้อมระบบ Triple Health Probes และ Connection Draining

02-sample-app/src/index.ts (ตัวจัดการสัญญาณ SIGTERM)

```
// ฟังก์ชันจัดการสัญญาณ SIGTERM เพื่อปลดระวางอย่างปลอดภัย
const handleSignal = (signal: string) => {
  isShuttingDown = true;
  isReady = false; // ปลดตัวออกจาก Ingress Endpoint ทันที

  server.close(async () => {
    // เคลียร์การเชื่อมต่อฐานข้อมูลและ Redis ที่ค้างอยู่
    await dbPool.end();
    await redis.quit();
    process.exit(0);
  });

  // วาล์วนิรภัย: บังคับปิดหากเคลียร์ไม่เสร็จภายใน 25 วินาที
  setTimeout(() => process.exit(1), 25000).unref();
};

process.on('SIGTERM', () => handleSignal('SIGTERM'));
```

ขั้นตอนการรันแล็บที่ละสเต็ป

```
cd workshop-setup/02-sample-app
# 1. รัน PostgreSQL + Redis + App ในเครื่อง (ทุก OS)
docker compose up -d --build

# 2. ทดสอบ Startup และ Readiness Probes
# สำหรับ macOS / Linux / Windows curl:
curl -i http://localhost:3000/healthz/startup
curl -i http://localhost:3000/healthz/readiness

# ทางเลือก Windows PowerShell:
Invoke-RestMethod http://localhost:3000/healthz/readiness

# 3. ตรวจสอบค่า Metrics สำหรับ Prometheus
curl -s http://localhost:3000/metrics | grep http_
```

ข้อคิดสำคัญ: เมื่อ `isReady = false` ระบบ Kubernetes จะหยุดส่งคำขอใหม่จากลูกค้าทันที ก่อนที่ `server.close()` จะเริ่มเคลียร์คำขอเดิมที่ค้างอยู่จนจบ!

กฎเหล็กระดับโปรดักชันและข้อควรระวังสำหรับ 12-Factor

หลักเกณฑ์สถาปัตยกรรมเพื่อการสร้างไมโครเซอร์วิสบนคลาวด์เนทีฟที่พร้อมสับเปลี่ยนและทนทานสูง

พฤติกรรมอันตรายที่ต้องหลีกเลี่ยง (Anti-Patterns)

- การฝังรหัสผ่านหรือไฟล์ `.env.production` ลงในเลเยอร์ของ Docker Image
- การเขียนไฟล์ที่ผู้ใช้แอปโหลดลง Local Disk ของคอนเทนเนอร์ (ข้อมูลจะหายเมื่อรีสตาร์ท)
- การละเลยสัญญาณ `SIGTERM` จนโดน K8s ฆ่าทิ้งอย่างรุนแรงด้วย `SIGKILL` ใน 30 วินาที
- การเขียนล็อกลงใน `/var/log/app.log` และตั้งค่า logrotate ข้างในคอนเทนเนอร์

หลักเกณฑ์ทางสถาปัตยกรรมที่แนะนำ

- Zero State:** ปฏิบัติต่อทุก Pod เสมือนพร้อมจะดับสูญได้ทุกวินาทีโดยไม่กระทบระบบ
- Probe Hygiene:** กำหนดคอนฟิกร์ `startup`, `readiness`, และ `liveness` probes ให้แยกจากกันอย่างชัดเจน
- Resource Envelopes:** ระบุ `requests` และ `limits` ของ CPU และหน่วยความจำเสมอ
- Structured Output:** ส่งออกล็อกเป็น JSON บัสดึงเดี่ยวตรงไปยัง stdout แบบ unbuffered

กับดักเชิงสังคมและเทคนิคในการก้าวสู่คลาวด์

ทำไมการลงทุนหลายล้านใน Kubernetes และ GitOps ถึงล้มเหลว หากโครงสร้างองค์กรยังทำงานแบบแยกส่วน (Silos)

รูปแบบความล้มเหลวมูลค่ามหาศาล

องค์กรทุ่มเวลา 6 เดือนสร้างคลัสเตอร์ Kubernetes ที่ทันสมัยที่สุดบน PROEN Cloud แต่การดีพลอยระบบยังคงต้องใช้เวลา 3 สัปดาห์, สาขาของโค้ดแยกตัวนานหลายเดือน และเมื่อระบบล่มก็เปิดจากประชุมเพื่อชี้แจงหาคนผิด

เพราะอะไร? เพราะองค์กรยังคงยึดติดกับโครงสร้างแบบเดิม: นักพัฒนาโยนโค้ดข้ามกำแพงให้ทีม Ops, ทีม QA ตรวจสอบด้วยมือนานนับสัปดาห์ และทีมความปลอดภัยเข้ามาตรวจเฉพาะขั้นตอนสุดท้าย

"หากคุณนำองค์กรที่มีโครงสร้างเตตระมาใช้งานไมโครเซอร์วิสและ Kubernetes สิ่งที่คุณจะได้ก็คือ องค์กรที่เตตระซึ่งรันไมโครเซอร์วิสและ Kubernetes อยู่เบื้องหลังเท่านั้น"

— หลักการของระบบเชิงสังคมและเทคนิค (Socio-Technical Systems)

ความจริงในเชิงสถาปัตยกรรม: โครงสร้างองค์กรและสถาปัตยกรรมซอฟต์แวร์คือเงาสะท้อนซึ่งกันและกัน คุณไม่สามารถยกระดับสิ่งใดสิ่งหนึ่งได้โดยไม่ปรับปรุงอีกสิ่งควบคู่กัน

กฎของ Conway และโครงสร้างทีมแบบ Team Topologies

การจัดโครงสร้างทีมงานให้สอดคล้องกับสถาปัตยกรรมแบบ Decoupled ของระบบ Cloud-Native

กฎของ Conway (Conway's Law - 1967)

"องค์กรที่ออกแบบระบบ ถูกจำกัดให้สร้างผลงานการออกแบบที่เป็นเสมือนสำเนาของโครงสร้างการสื่อสารภายในองค์กรนั้นๆ"

กลยุทธ์ Inverse Conway Maneuver: ปรับเปลี่ยนโครงสร้างทีมให้เป็นทีมขนาดเล็กที่ทำงานแบบ Cross-Functional และมีอิสระในการส่งมอบ เพื่อขับเคลื่อนสถาปัตยกรรมซอฟต์แวร์สู่ไมโครเซอร์วิสที่แยกส่วนกันอย่างแท้จริง

Stream-Aligned Teams (ทีมมุ่งเน้นสายงาน)

ดูแลระบบตั้งแต่ต้นจนจบตามโดเมนธุรกิจ (ระบบชำระเงิน, ออกตัว, ตรวจสอบบัตร)

Platform Engineering Team (ทีมแพลตฟอร์ม)

ให้บริการแพลตฟอร์มแบบ Self-Service: PROEN Cloud, Kubernetes, Argo CD, Observability

Enabling Teams (ทีมเสริมศักยภาพ)

สถาปนิกคลาวด์และผู้เชี่ยวชาญ DevOps ที่คอยยกระดับทักษะ GitOps และความปลอดภัย

Complicated-Subsystem Teams (ทีมระบบซับซ้อนเฉพาะทาง)

ดูแลอัลกอริทึมเฉพาะทาง เช่น การเข้ารหัส QR Code หรือเอนจินจัดสรรที่นั่ง

ตัวชี้วัด DORA และการวิเคราะห์หาสาเหตุที่แท้จริงแบบไม่โทษกัน

เปลี่ยนอุปนิสัยการณัของระบบลุ่มให้เป็นวิวัฒนาการเชิงโครงสร้าง ด้วยเทคนิค 5 Whys และบันทึกการตัดสินใจ ADR

การวิเคราะห์หาสาเหตุด้วยเทคนิค 5 Whys (RCA)

1. ทำไมระบบชำระเงินจึงตอบกลับด้วยข้อผิดพลาด 500?

Connection Pool ของฐานข้อมูลเต็ม

2. ทำไม Connection Pool จึงเต็ม?

ทุกๆ คำขอสั่งเปิด Connection ใหม่โดยไม่ปล่อยคืน

3. ทำไมถึงไม่ปล่อยคืน Connection?

Unhandled Promise Rejection ในโค้ดคืนเงิน ข้ามคำสั่ง `client.release()`

4. ทำไมกรณีนี้ถึงหลุดรอดไปถึงโปรดักชัน?

Integration Tests ขาดชุดทดสอบสำหรับเคสคำสั่งซื้อถูกยกเลิก

5. ทำไมถึงขาดชุดทดสอบนี้?

ไปป์ไลน์ CI ขาดการบังคับใช้เกณฑ์ Contract-Testing Gates

บันทึกการตัดสินใจทางสถาปัตยกรรม (ADR Protocol)

ทุกการตัดสินใจทางสถาปัตยกรรมต้องบันทึกลงใน Git ควบคู่กับโค้ด:

- **ชื่อ:** ADR-0012: บังคับใช้ PgBouncer Transaction Pooling
- **บริบท:** การแย่งชิงตัวช่วงกราฟฟิสูงทำให้ Connection ของ Postgres เต็ม
- **การตัดสินใจ:** ติดตั้ง PgBouncer คั่นกลางระหว่าง Pods กับ PostgreSQL
- **ผลสืบเนื่อง:** ต้องปิดการใช้งาน Named Prepared Statements

กฎทอง: Blameless RCA จะไม่ถามว่า "ใครทำพลาด?" แต่จะถามว่า "ระบบของเรามีช่องโหว่ตรงไหนที่ยอมให้ความผิดพลาดนี้เกิดขึ้นได้?"

รายการตรวจสอบวัฒนธรรมวิศวกรรมสู่ความเป็นเลิศ

ระเบียบปฏิบัติที่ขาดไม่ได้สำหรับองค์กรวิศวกรรมคลาวด์ที่มีประสิทธิภาพสูง

ไม่ชี้นิ้วโทษกัน

การทบทวนอุบัติการณ์

มุ่งเน้นการสร้างเกราะป้องกันเชิงระบบและเพิ่มชุดทดสอบอัตโนมัติ

ส่งมอบทุกวัน

การปล่อยงานชุดเล็ก

ส่งมอบการเปลี่ยนแปลงขนาดเล็กเพื่อลดความเสี่ยงและปัญหาข้อขัดแย้งของโค้ด

บันทึก ADR

การทำกับดักผ่าน GIT

บันทึกทุกการตัดสินใจทางสถาปัตยกรรมครั้งสำคัญลงใน Git ในรูปแบบ ADR

Self-Service

แพลตฟอร์มภายใน

ทีมแพลตฟอร์มสร้าง API และเกมเพลตเพื่อให้ทีมพัฒนาส่งมอบงานได้อย่างอิสระ

สรุปด้านวัฒนธรรม: การส่งมอบซอฟต์แวร์ระดับชั้นนำจะเกิดขึ้นได้เมื่อมีความปลอดภัยทางจิตวิทยา (Psychological Safety), การทำกับดัก ADR ที่โปร่งใส และการสร้างชุดตรวจสอบอัตโนมัติอย่างต่อเนื่อง

วิกฤตแย่งซื้อบัตรตอน 10:00:00 น.

เมื่อกราฟฟิคพุ่งสูงในแนวตั้งจนทำให้เกิดการขายตั๋วเกิน, เกิดเดดล็อก และระบบล่มเป็นลูกโซ่

ความเสียหายรูปแบบที่ 1

การขายของเกินโควตา (Race Conditions)

มีคำสั่งซื้อ 500 รายการยังเข้ามาพร้อมกันเพื่ออ่าน `SELECT stock FROM tickets` ทุกคำขออ่านได้ `stock = 1` และเดินทางตัดเงิน ทำให้ขายบัตรเกินจำนวนที่นับจริงไปถึง 499 ใบ

ความเสียหายรูปแบบที่ 2

การหมดสภาพของ Thread แบบลูกโซ่

ช่องทางชำระเงินภายนอกตอบสนองช้าลงจาก 200ms เป็น 12 วินาที ไมโครเซอร์วิสฝั่งต้นทางถือ HTTP Connection ค้างไว้ ส่งผลให้ Connection Pool กั๊กระบบตายสนิท

ความเสียหายรูปแบบที่ 3

ปรากฏการณ์แคชแตก (Cache Stampede)

แคชของฝั่งที่นิ่งหมดอายุพร้อมกันในวินาทีที่ 10:00:00 ผู้ใช้ 10,000 คน จึงยังตรงเข้าสู่ฐานข้อมูลหลักพร้อมๆ กันเพื่อคำนวณ JSON ชุดเดิม จนฐานข้อมูลล่ม

ภารกิจทางสถาปัตยกรรม: สร้างกลไกล็อก Concurrency ทางคณิตศาสตร์, ติดตั้ง Circuit Breakers และป้องกัน Cache Stampede ด้วยความน่าจะเป็น

การควบคุม Concurrency, Saga Pattern และ Circuit Breakers

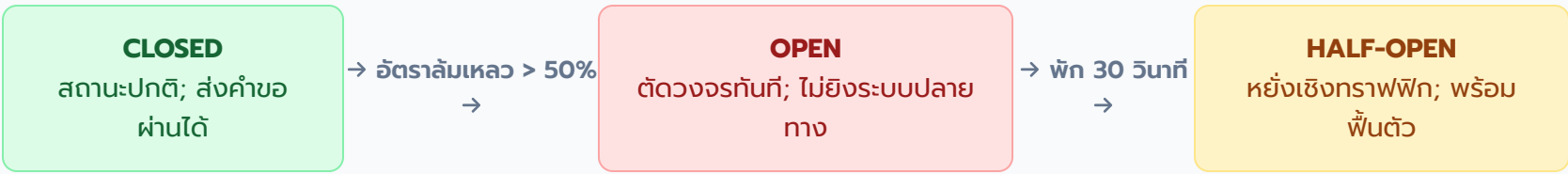
รูปแบบทางสถาปัตยกรรมเพื่อความถูกต้องแม่นยำของธุรกรรมการเงิน แม้ในช่วงกราฟฟิคทะเลาะหาศาล

Pessimistic Row-Level Locking (PostgreSQL)

คำสั่ง `SELECT ... FOR UPDATE` จะล็อกเฉพาะแถวของสต็อกที่กำลังทำการกายไต้ ACID Transaction ทำให้คำขออื่นๆ ต้องเข้าคิวรออย่างเป็นระเบียบ จึงป้องกันการขายเกินได้อย่างสมบูรณ์

Saga Pattern เทียบกับ Distributed 2PC

Two-Phase Commit (2PC) ไม่สามารถสเกลบนไมโครเซอร์วิสได้ Saga จึงเข้ามาบริหารธุรกรรมข้ามเซอร์วิสผ่านชุดของ Local Transactions ควบคู่กับ **ธุรกรรมชดเชย (Compensating Transactions)** เมื่อเกิดข้อผิดพลาด



กลไก Circuit Breaker: ช่วยปกป้อง Payment Gateways และ API ปลายทาง ไม่ให้ถูกทำลายจนล่มสลายเมื่อเกิดวิกฤตกราฟฟิคฝั่งต้นทาง

ทดสอบความเครียดและล็อกแถวข้อมูล PostgreSQL

จำลองคำสั่งซื้อตั๋ว 30 รายการพร้อมกันในเสี้ยววินาที เพื่อทดสอบระบบจองบัตรบน PostgreSQL ของเรา

O2-sample-app/src/index.ts (หัวใจของ Row-Lock)

```
await client.query('BEGIN');

// ล็อกแถวข้อมูลแบบ Exclusive ป้องกันการจองซ้อนทับ
const res = await client.query(`
  SELECT total_capacity, issued_count
  FROM ticket_tiers WHERE id = $1 FOR UPDATE;
`, [tierId]);

if (res.rows[0].issued_count ≥ res.rows[0].total_capacity) {
  throw new ConflictException('บัตรหมดแล้ว (Sold Out)');
}

await client.query(`
  UPDATE ticket_tiers
  SET issued_count = issued_count + 1 WHERE id = $1;
`, [tierId]);

await client.query('COMMIT');
```

รันสคริปต์ทดสอบการทำงานพร้อมกัน

```
cd workshop-setup/06-chaos-and-verification

# macOS & Linux:
chmod +x test-concurrency-locks.sh && ./test-concurrency-locks.sh

# Windows (PowerShell):
.\test-concurrency-locks.ps1

# ผลลัพธ์ที่ได้ (เหมือนกันทุกระบบ):
Firing 30 parallel checkout requests...
[CONFIRMED ORDERS (201)]: 30
[SAFELY SOLD OUT (409)]: 0
[SYSTEM FAILURES (500)]: 0
SUCCESS: ป้องกันการขายเกิน 100% และไม่มีข้อผิดพลาด 500!
```

ข้อคิด: แม้ว่าคำสั่งซื้อ 30 รายการจะพุ่งเข้ามาในมิลลิวินาทีเดียวกัน แต่การล็อกระดับแถวของ PostgreSQL ช่วยป้องกันปัญหา Race Condition ได้อย่างสมบูรณ์แบบ

รายการตรวจสอบความทนทานต่อกราฟฟิคมหาศาล

กฎเหล็กเพื่อการรับมือช่วงเปิดขายบัตรคอนเสิร์ตหรือแคมเปญ Flash Sales ขนาดใหญ่

ล็อกระดับแถว

FOR UPDATE

ล็อกแถวของสินค้าคงคลังใน Atomic DB Transaction เสมอ อย่าใช้ UPDATE แบบไม่มีเงื่อนไข

PgBouncer

รวม POOL เชื่อมต่อ

ติดตั้ง PgBouncer ในโหมด Transaction Pooling เพื่อควบคุมขีดจำกัด Connection ของ Postgres

XFetch

ป้องกันแคชแตก

ใช้อัลกอริทึม Early Expiration ด้วยความน่าจะเป็น เพื่อให้มีเพียง 1 Thread เข้าไปคำนวณแคชใหม่

Pre-Scale

วอร์มรันไทม์ล่วงหน้า

ขยาย Pod ล่วงหน้า 30 นาทีก่อนเปิดขายตั๋ว เพราะ HPA แบบ Reactive ตอบสนองช้าเกินไป

สรุปการขยายระบบ: การสเกลระบบไม่ใช่การทุ่มแรมแบบไม่จำกัด แต่คือการกำจัดการแย่งชิงทรัพยากรส่วนกลาง และบังคับใช้การจัดลำดับ Transaction ที่เข้มงวด

ม้าโทรจันในคอนเทนเนอร์อิมเมจของคุณ

การโจมตีห่วงโซ่อุปทานยุคใหม่สามารถเจาะคลัสเตอร์โปรดักชันได้ โดยที่ซอร์สโค้ดของคุณไม่มีบีกแม้แต่บรรทัดเดียว

ภัยคุกคามในโค้ดโอเพ่นซอร์สที่เราพึ่งพา

แอปพลิเคชันยุคใหม่ประกอบด้วย **โค้ดภายนอกและเลเยอร์ฐานของ OS** มากถึง **90%** ผู้ไม่ประสงค์ดีจึงไม่จำเป็นต้องเจาะผ่านไฟร์วอลล์ แต่หันมาวางยาพิษในห่วงโซ่อุปทานซอฟต์แวร์แทน:

- **Base Images แฝงช่องโหว่:** อิมเมจทางการที่ฝังช่องโหว่ CVE ของ OpenSSL หรือ glibc ที่ยังไม่ได้รับการแพตช์
- **การสวมรอยแพ็คเกจ npm:** การตั้งชื่อแพ็คเกจคล้ายของจริงเพื่อหลอกให้ดาวน์โหลดโค้ดอันตราย
- **การแฮก CI Runner:** ผู้ไม่ประสงค์ดีแอบแทรกโค้ดอันตรายในขั้นตอนการคอมไพล์ไบนารี (เช่น SolarWinds, Codecov)

ความจำเป็นของหลักการ Zero-Trust

ในสถาปัตยกรรมแบบ Zero-Trust เราไม่สามารถไว้วางใจคอนเทนเนอร์เพียงเพราะมันถูกเก็บอยู่ใน Private Registry ได้ เราต้องเรียกร้อง **หลักฐานยืนยันทางคณิตศาสตร์ที่พิสูจน์ได้:**

- หลักฐานที่มาของคอมมิตต้นฉบับ (Provenance)
- บัญชีรายการส่วนประกอบซอฟต์แวร์ทั้งหมด (SBOM)
- ลายมือชื่อดิจิทัลที่สร้างโดย CI Runner ที่ได้รับการยืนยันตัวตนแล้ว

เฟรมเวิร์ก SLSA และคอนเทนเนอร์ Distroless

การลดพื้นที่ผิวการถูกโจมตี (Attack Surface) และการบังคับใช้การตรวจสอบที่มาตั้งแต่ต้นน้ำถึงปลายน้ำ

SLSA ระดับ 1: สร้างระบบอัตโนมัติ

การบิลด์เป็นไปโดยอัตโนมัติพร้อมสร้าง Metadata ที่ตรวจสอบได้

SLSA ระดับ 2: รับรองที่มาด้วยลายมือชื่อ

ระบบบิลด์บนคลาวด์สร้างเอกสารรับรองที่เข้ารหัสด้วยลายมือชื่อดิจิทัล

SLSA ระดับ 3: รันเนอร์แยกส่วนแบบชั่วคราว

การบิลด์แบบ Hermetic ป้องกันการดัดแปลงแก้ไขข้ามโปรเจกต์

SLSA ระดับ 4: ตรวจสอบสองคนและการบิลด์ซ้ำได้

การันตีว่าสามารถบิลด์ซ้ำได้ผลลัพธ์เดิม พร้อมการลงนามอนุษัติของเพื่อนร่วมงาน

กลยุทธ์ Google Distroless

อิมเมจลินุกซ์ทั่วไป (Ubuntu, Debian, Alpine) จะแถม Shell (`/bin/sh`), แพ็กเกจเมเนเจอร์ (`apt` , `apk`) และเครื่องมือเน็ตเวิร์ก (`curl` , `wget`) มาด้วย

Distroless จะมีเฉพาะตัวไบনারีของแอปพลิเคชันเท่านั้น หากผู้โจมตีเจาะแอปได้ ก็จะไม่มี Shell ให้เปิดใช้งาน และไม่มีแพ็กเกจเมเนเจอร์ให้ดาวน์โหลดมัลแวร์ตัวอื่นเข้ามาเพิ่มได้เลย

สร้าง SBOM, สแกนช่องโหว่ด้วย Trivy และเซ็นรับรองด้วย Cosign

ผลิตเอกสาร CycloneDX SBOM, ตรวจสอบช่องโหว่ CVE และลงลายมือชื่อดิจิทัลบน OCI Artifacts

03-supply-chain/secure-build-and-sign.(sh | ps1)

```
# 1. บิลด์คอนเทนเนอร์ฐานแบบ Distroless
docker build -t order-service:1.3.0 .

# 2. ผลิตเอกสาร CycloneDX SBOM ด้วย Syft
syft order-service:1.3.0 -o cyclonedx-json > sbom.json

# 3. สแกนหาช่องโหว่ระดับวิกฤต (Critical) ด้วย Trivy
trivy image --severity CRITICAL --exit-code 1 order-service:1.3.0

# 4. ลงลายมือชื่อดิจิทัลด้วย Cosign
cosign generate-key-pair
cosign sign --key cosign.key order-service:1.3.0
cosign attest --key cosign.key --predicate sbom.json order-service:1.3.0
```

รันสคริปต์ทดสอบความปลอดภัยห่วงโซ่อุปทาน

```
cd workshop-setup/03-supply-chain

# macOS & Linux:
chmod +x secure-build-and-sign.sh && ./secure-build-and-sign.sh

# Windows (PowerShell):
.\secure-build-and-sign.ps1

# ผลลัพธ์การรัน (ทุกแพลตฟอร์ม):
[OK] Container built successfully.
[OK] SBOM generated at: sbom.cyclonedx.json
[OK] Vulnerability scan passed.
[OK] Signature verified against cosign.pub
```

การบังคับใช้บน K8s: เมื่อติดตั้ง Kyverno Admission Controller คอนเทนเนอร์ที่ไม่มีลายมือชื่อรับรองจะถูกปฏิเสธตั้งแต่หน้าประตู API Server ด้วยรหัส 403 กันที!

รายการตรวจสอบความปลอดภัยของห่วงโซ่อุปทานซอฟต์แวร์

มาตรการความปลอดภัยที่จำเป็นสำหรับการรับประกันความสมบูรณ์ของคอนเทนเนอร์และแพ็คเกจ

Distroless

ปราศจาก SHELL

ตัดระบบแพ็คเกจและ Shell ทั้งหมดออกจากคอนเทนเนอร์
โปรดักชัน

Non-Root

UID 65532

รันคอนเทนเนอร์ด้วยสิทธิ์ผู้ใช้ทั่วไปและเปิดโหมด Read-
Only Root Filesystem เสมอ

CycloneDX

การรับรอง SBOM

แนบเอกสาร SBOM ที่เซ็นรับรองทางดิจิทัลควบคู่ไปกับทุก
Release ใน Registry

Admission

KYVERNO / OPA

บล็อกคอนเทนเนอร์ที่ไม่ได้เซ็นรับรองหรือไม่ผ่านการสแกน
ไม่ให้ดีพลอยขึ้นโปรดักชัน

สรุป DevSecOps: ความปลอดภัยไม่ใช่ขั้นตอนการตรวจด้วยมือในวันสุดท้ายของสปรินต์อีกต่อไป แต่เป็นคุณสมบัติทางคณิตศาสตร์ที่ตรวจสอบได้โดยอัตโนมัติในไปป์ไลน์ CI/CD

ภาษี YAML ของนักพัฒนา (The Developer 'YAML Tax')

ทำไมการบังคับให้นักพัฒนาซอฟต์แวร์ต้องเขียน Kubernetes Manifests จึงดูเร่งความเร็วของธุรกิจ

ภาระงานที่ตกโถมสู่นักพัฒนาผลิตภัณฑ์

Kubernetes มอบพลังอันมหาศาลให้กับงาน Operation แต่กลับสร้างภาระทางความคิด (Cognitive Load) อย่างมหาศาลให้กับนักพัฒนา:

- นักพัฒนาต้องเสียเวลา **30%–40%** ไปกับการเขียน **Dockerfile**, แก้ไขปัญหาแคชเลเยอร์ และคอนฟิก Ingress YAML
- แต่ละทีมเขียนการตั้งค่า Docker ที่แตกต่างกันตามความเคยชิน ทำให้ขาดมาตรฐาน
- ทีมความปลอดภัยต้องเหนื่อยล้ากับการตรวจสอบ Dockerfile นับร้อยไฟล์ว่าใช้ Base OS ที่ถูกรุ่นหรือไม่

"นี่คือซอร์สโค้ดของฉัน... ช่วยรับมันบนคลาวด์ให้ที ฉันไม่สนใจหรอกว่าเบื้องหลังจะทำอย่างไร"

— **อุดมคติของงานวิศวกรรมแพลตฟอร์ม (Platform Engineering)**

การบรรยายพิเศษ: VMware by Broadcom (คุณชาลี คัมภีรภาพ) นำเสนอเลเยอร์ Platform Abstraction ยุคใหม่ที่จะเข้ามาขจัดช่องว่างนี้

Paketo Buildpacks และ VMware Tanzu Platform

สร้าง OCI Container มาตรฐานสำหรับโปรดักชันได้โดยตรงจากซอร์สโค้ด โดยไม่ต้องเขียน Dockerfile

1. ตรวจจับภาษาอัตโนมัติ (Source Detection)

Buildpack วิเคราะห์โค้ด Node.js / Java / Go และ Dependencies อัตโนมัติ

2. จัดเลเยอร์ OS ที่ปลอดภัย (Hardened OS Layer)

ดึงเลเยอร์ระบบปฏิบัติการ Ubuntu Jammy ที่ผ่านการรับรองความปลอดภัยมาใช้งาน

3. คอมไพล์รันไทม์และแอปพลิเคชัน

คอมไพล์ Bytecode ควบคู่กับการตั้งค่าระบบแบบ Non-Root ที่ได้รับการปรับแต่งมาอย่างดี

4. ส่งออกเป็น OCI Image มาตรฐาน

ผลิตคอนเทนเนอร์ OCI ตามมาตรฐานสากลพร้อมเอกสาร SBOM ในตัว

พลังพิเศษของการ Rebase เลเยอร์

Dockerfile ทั่วไปจะหลอมรวม OS และแอปพลิเคชันเข้าด้วยกัน หาก OS มีช่องโหว่ แอป 500 ตัวจะต้องเสียเวลาคอมไพล์ใหม่ทั้งหมด

ด้วย **Paketo Buildpacks** เลเยอร์ของ OS จะถูกแยกออกจากกันอย่างเด็ดขาด คุณสามารถสลับเลเยอร์ OS ที่อุดช่องโหว่แล้วให้กับคอนเทนเนอร์ทุกตัวได้ในเสี้ยววินาทีด้วยคำสั่ง `pack rebase` โดยไม่ต้องแตะต้องหรือคอมไพล์โค้ดใหม่เลย!

สร้างอิมเมจจากซอร์สโค้ดและ Rebase ระบบปฏิบัติการทันที

สร้าง OCI Container จากซอร์สโค้ดโดยตรง และทำการสลับเลเยอร์ OS ทันทีโดยไม่ต้องคอมไพล์ใหม่

คำสั่งเทอร์มินัล: Pack CLI

```
# 1. บิลด์ OCI Container ระดับโปรดักชันจากซอร์สโค้ดโดยตรง
# สังเกต: ไม่จำเป็นต้องมีไฟล์ Dockerfile เลย!
pack build registry.proen.cloud/gvents/order-service:1.3.0 --builder paketobuildpacks/builder-jammy-base --path ../02-sample-app

# 2. ตรวจสอบเลเยอร์ของ OCI Container ที่ถูกสร้างขึ้น
docker inspect registry.proen.cloud/gvents/order-service:1.3.0

# 3. จำลองการอัปเดตแพตช์ความปลอดภัยของ OS:
# สลับตัวชี้เลเยอร์ได้ในเวลาไม่ถึง 100 มิลลิวินาที!
pack rebase registry.proen.cloud/gvents/order-service:1.3.0

# ผลลัพธ์:
Successfully rebased to new base layer! Zero recompilation!
```

การประกาศ Tanzu Workload CRD

บน Tanzu Application Platform นักพัฒนาเพียงแค่ส่งมอบไฟล์ **Workload** ขนาดกะทัดรัดเข้าสู่ Git:

```
apiVersion: carto.run/v1alpha1
kind: Workload
metadata:
  name: order-service
spec:
  source:
    git:
      url: https://github.com/3lVv0w/code-to-cloud-workshop.git
      ref: { branch: main }
```

รายการตรวจสอบ Platform Abstraction ในองค์กร

แยกผลิตภาพของนักพัฒนาออกจากงานก่อสร้างระบบโครงสร้างพื้นฐาน

Zero

DOCKERFILES

สร้างมาตรฐานการผลิตคอนเทนเนอร์ด้วย Cloud Native Buildpacks ในทุกทีม

< 100ms

OS REBASE

อัปเดตแพตช์ช่องโหว่ OS ในคอนเทนเนอร์นับพันตัวได้โดยไม่ต้องบิลด์โค้ดใหม่

มาตรฐาน

OCI ARTIFACTS

ผลิตคอนเทนเนอร์อิมเมจมาตรฐานที่นำไปรันบนคลัสเตอร์ Kubernetes ใดก็ได้

Cartographer

ห่วงโซ่อุปทาน

ทำงานอัตโนมัติในส่วน Outer Loop: สแกนความปลอดภัย, เซ็นอิมเมจ และเราท์ Ingress

สรุปปรับใหม่: มอบพลังให้นักพัฒนามุ่งเน้นที่ Business Logic ขณะที่ระบบอัตโนมัติของแพลตฟอร์มช่วยรับประกันความปลอดภัยและมาตรฐานคลาวด์

ปริศนาระบบล่มตอนที่ 3: ล็อกไว้โครงสร้าง

ทำไมข้อความล็อกแบบ Plaintext ทั่วไปถึงไร้ประโยชน์โดยสิ้นเชิงในสถาปัตยกรรมไมโครเซอร์วิสแบบกระจาย

ความมืดมิดในระบบแบบกระจาย (The Fog of Systems)

วิศวกร On-Call ได้รับข้อความแจ้งเตือนตอนดึก: "อัตราความล้มเหลวของการชำระเงินพุ่งสูงถึง 25%"

พวกเขา SSH เข้าสู่เซิร์ฟเวอร์เพื่อเปิดไฟล์ล็อก และต้องเจอกับข้อความตัวอักษรนับหมื่นบรรทัดต่อวินาที ข้อผิดพลาดฟองที่ API Gateway แต่สาเหตุที่แท้จริงกลับเป็น Timeout เจียบๆ ใน Inventory Service ที่เกิดจากการรอคิวล็อกของฐานข้อมูล

หากไม่มี **Correlation ID** การไล่รอย **Transaction** เดียวข้าม 5 ไมโครเซอร์วิสอาจต้องใช้เวลาหลายชั่วโมง

Observability เทียบกับ Monitoring ทั่วไป

Monitoring: บอกคุณว่าระบบ กำลังพัง (เช่น อัตราความผิดพลาด HTTP 500 > 10 ครั้ง)

Observability: ช่วยให้คุณเข้าใจว่าระบบพัง เพราะอะไร โดยการชักตามสถานะภายในจากสัญญาณทางไกล (Metrics, Logs, และ Traces)

เป้าหมายของเรา: พสานสัญญาณทางไกลทั้งหมดเข้าด้วยกันผ่าน `trace_id` ตามมาตรฐาน W3C ข้ามทุกบริการบน PROEN Cloud

สามเสาหลัก Observability และการส่งมอบแบบ Canary

การติดตั้ง OpenTelemetry, การวัดผลแบบ RED และการสลับกราฟฟิคขึ้นระบบอย่างไร้ความเสี่ยง

METRICS

Prometheus RED Method

Rate (คำขอ/วินาที), **Errors** (รหัส 5xx), และ **Duration** (เวลาแฝง P95/P99)

STRUCTURED LOGS

Loki JSON Streams

ทุกล็อกพ่นออกมาเป็น JSON ที่มีฟิลด์ `trace_id` และ `span_id` ทำกับเสมอ

DISTRIBUTED TRACES

OpenTelemetry & Tempo

ส่งต่อ Header `traceparent` ตามมาตรฐาน W3C ข้ามขอบเขตเครือข่ายทุกจุด

การปล่อยระบบแบบก้าวหน้า (Argo Rollouts Canary)

อย่าปล่อยโค้ดเวอร์ชันใหม่ให้ผู้ใช้ 100% พร้อมกัน ให้ค่อยๆ เพิ่มสัดส่วนกราฟฟิคทีละขั้น:

- 1 กราฟฟิค 10% • ตรวจสอบ Error Rate นาน 5 นาที
- 2 กราฟฟิค 25% • ตรวจสอบวัดเวลาแฝง P99 นาน 10 นาที
- 3 กราฟฟิค 50% • ยืนยันว่าไม่มีบักตกค้าง
- 4 ปล่อย 100% เต็ม • ปลดระวาง Pod รุ่นเก่าทิ้ง

ตั้งค่า Canary Rollout บน Argo Rollouts และการแจ้งเตือน Prometheus

กำหนดค่า Argo Rollouts พร้อมระบบวิเคราะห์ Metrics อัตโนมัติ และกฎแจ้งเตือนตามเกณฑ์ SLO

ข้อกำหนด Argo Rollouts Canary

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: order-service
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 5m }
  analysis:
    templates:
      - templateName: error-rate-check
# หากอัตราความผิดพลาด > 1% ให้สับกลับ Rollout ทันที!
```

กฎตรวจจับ Error Rate สูงบน Prometheus

```
- alert: HighHttpErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[2m]))
    /
    sum(rate(http_requests_total[2m])) > 0.01
  for: 2m
  labels: { severity: critical }
  annotations:
    summary: "API Error rate exceeded 1% on {{ $labels.service }}"
```

การย้อนกลับอัตโนมัติ: หาก Prometheus แจ้งเตือน `HighHttpErrorRate` ระหว่างทดสอบ Canary ระบบจะสั่งย้อนกราฟฟิกลับสู่เวอร์ชัน Stable 100% ในทันที!

แนวปฏิบัติที่ดีที่สุดด้าน Observability และการนำขึ้นระบบ

Golden Signals และหลักการส่งมอบระบบเพื่อความมั่นคงปลอดภัยบนโปรดักชัน

P95 / P99

เปอร์เซ็นต์ไทล์

อย่าพึ่งพาค่าเฉลี่ย (Mean) เพราะค่าเฉลี่ยจะบดบังปัญหา เวลาแฝงที่รุนแรงของผู้ใช้บางส่วน

W3C

บริบท TRACE

ส่งต่อ `traceparent` headers ข้ามทุกขอบเขตการเรียกใช้งาน HTTP, gRPC และ Message Queue

JSON

เลือกแบบมีโครงสร้าง

เลือกทุกบรรทัดต้องเป็นรูปแบบ JSON เพื่อการทำดัชนีและค้นหาที่รวดเร็วบน Grafana Loki

Canary

จำกัดผลกระทบ

จำกัดความเสียหายของการปล่อยระบบที่มีข้อผิดพลาด ให้กระทบผู้ใช้กลุ่มแรกไม่เกิน 10%

สรุปการดีพลอย: การขึ้นระบบจะไม่ใช้งานลูกระกียามตึกอีกต่อไป แต่ด้วย Canary Analysis และ OpenTelemetry การปล่อยงานจะกลายเป็นเรื่องปกติและตรวจสอบความถูกต้องได้ในตัวเอง

ฝันร้ายของ 'Merge Hell'

ทำไมการแยก Feature Branch ยาวนานหลายเดือนตามแบบ GitFlow ดั้งเดิม ถึงทำลายความเร็วและทำให้เกิดบ่ก้ำซ้ำซ้อน

กับดักของ GitFlow แบบดั้งเดิม

ทีมพัฒนาทำงานตามระเบียบเดิม: แยกกิ่ง feature , develop , และ release กิ่งไว้ยาวนานหลายเดือน

ปัญหาที่เกิดขึ้นตามมา:

- ทีม A แก้ไขระบบสั่งซื้ออยู่นาน 6 สัปดาห์
- ทีม B ปรับโครงสร้างโมเดลฐานข้อมูลควบคู่กันไป
- เมื่อทีม A นำโค้ดมาเมิร์จ พวกเขาต้องเจอกับ **Merge Conflicts** มากถึง **140 จุด** ข้าม 80 ไฟล์
- วิศวกรต้องเสียเวลา 4 วันแก้งานด้วยมือ และเฟลอลบโค้ดที่แก้บ่ก้ำไปแล้วทิ้งไปโดยไม่ตั้งใจ

ในโลกของ GitOps นั้น Git คือสถานะของโปรดักชัน

ในสถาปัตยกรรม GitOps นั้น Git ไม่ใช่แค่ที่ฝากโค้ด แต่คือ **หลักฐานการตรวจสอบที่เข้ารหัสของโครงสร้างพื้นฐานจริง**

วินัยการใช้ Git ที่หละหลวม—คอมมิตที่ไม่มีคนรีวิว, กิ่งที่ค้างนาน, หรืออีเมลผู้เขียนปลอม—จะทำลายความน่าเชื่อถือของทั้งระบบการดีพลอย

Trunk-Based Development และการออกรุ่น SemVer อัตโนมัติ

รวมการเปลี่ยนแปลงขนาดเล็กเข้าสู่สายหลัก (Main) ทุกวัน พร้อมระบบ Semantic Versioning อัตโนมัติ

สายหลัก main (Trunk)

พร้อมนำขึ้นระบบเสมอ • ตรวจสอบผ่าน CI ต่อเนื่อง • สะท้อนสถานะโปรดักชัน

↑ Squash & Merge (วันละหลายรอบ)

ก้งงานอายุสั้น (< 24 ชั่วโมง)

การเปลี่ยนแปลงขนาดเล็ก • ผ่านชุดทดสอบอัตโนมัติ • ได้รับการรีวิวจากเพื่อนร่วมงาน

แยกการดีพลอยออกจากการปล่อยฟีเจอร์: ใช้ **Feature Flags** เพื่อเมิร์จโค้ดที่ยังทำไม่เสร็จเข้าสู่ `main` ได้อย่างปลอดภัยโดยที่ผู้ใช้อยังมองไม่เห็น

มาตรฐานข้อความ Conventional Commits

รูปแบบข้อความคอมมิตที่มีโครงสร้างจะช่วยขับเคลื่อนการคำนวณเวอร์ชัน SemVer อัตโนมัติ:

- `feat(orders): add Apple Pay pass support`
→ ปรับเลขรุ่นย่อย **MINOR** (v1.2.0 → v1.3.0)
- `fix(scanner): correct QR scan timeout threshold`
→ ปรับเลขแก้บั๊ก **PATCH** (v1.2.0 → v1.2.1)
- `feat(auth)!: replace legacy API keys with OAuth2`
→ ปรับเลขรุ่นหลัก **MAJOR** (v1.2.0 → v2.0.0)

เซ็นคอมมิตด้วยกุญแจ SSH และการกรองเส้นทางใน Monorepo

ตั้งค่าการลงลายมือชื่อดิจิทัลบน Git Commit และเวิร์กโฟลว์ Path-Filtering สำหรับ Monorepo

การตั้งค่าเซ็นคอมมิตด้วยกุญแจ SSH

```
# 1. ตั้งค่าให้ Git ใช้กุญแจ SSH ในการลงลายมือชื่อคอมมิต
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true

# 2. ทำการคอมมิตที่สามารถตรวจสอบที่มาได้
git commit -m "feat(security): enforce MFA on organizer login"

# 3. ตรวจสอบลายมือชื่อในหน้าจอคอนโซล
git log --show-signature -1

# ผลลัพธ์:
Good "git" signature for user@company.com with ED25519 key...
```

การกรองเส้นทางใน Monorepo บน CI

หลีกเลี่ยงการบิลด์ 20 ไมโครเซอร์วิสใหม่ เมื่อแก้ไขเพียงแคไฟล์เอกสาร:

```
on:
  push:
    branches: [main]
    paths:
      - 'workshop-setup/02-sample-app/**'
      - 'package.json'
```

เกราะป้องกัน Branch Protection: บล็อก Pull Request กันที่หากคอมมิตไม่มีลายมือชื่อรับรอง หรือไม่ผ่านชุดทดสอบอัตโนมัติบน CI

รายการตรวจสอบการกำกับดูแล Git ในระดับองค์กร

กฎเกณฑ์ที่ต้องยึดถืออย่างเคร่งครัดเพื่อการจัดการคลังโค้ดด้วยความเร็วระดับ Cloud-Native

< 24 ชม.

อายุถึงการทำงาน

ลบ Feature Branch ทันทีหลังจากทำ Squash-Merge
เข้าสู่ `main` เรียบร้อยแล้ว

SSH / GPG

เซ็นรับรองคอมมิต

บล็อกคอมมิตที่ไม่มีลายมือชื่อ เพื่อรับรองตัวตนผู้เขียนตาม
มาตรฐานความปลอดภัย SOC2/ISO

Linear

ประวัติ GIT เป็นเส้นตรง

บังคับใช้ Squash-and-Merge เพื่อขจัด Merge
Commit ที่ว่างเปล่าและช่วยให้ทำ Git Bisect ได้ง่าย

2 บุคคล

การตรวจสอบโค้ด

กำหนดให้มีวิศวกรระดับซีเนียร์อนุมัติอย่างน้อย 2 คนก่อน
เมิร์จขึ้นสู่กิ่งที่มีการป้องกัน

สรุปวินัย Git: การทำ GitOps ที่รวดเร็วต้องการสุขอนามัยในการใช้ Git ที่ดีเยี่ยม หากขั้นตอนการทำงานบน Git สะเปะสะปะ คลัสเตอร์โปรดักชันของคุณก็จะวุ่นวายไม่แพ้กัน

อันตรายจากการฝาก Credential คลัสเตอร์ไว้ใน CI ภายนอก

ทำไมการมอบสิทธิ์ cluster-admin ให้กับ Jenkins หรือ GitHub Actions จึงละเมิดหลักการ Zero Trust อย่างรุนแรง

ช่องโหว่ความปลอดภัยของระบบ CI แบบ Push ดั้งเดิม

ในระบบ CI/CD แบบเก่า นักพัฒนามักเก็บไฟล์ `kubeconfig` ที่มีสิทธิ์ `cluster-admin` สูงสุดไว้ใน GitHub Secrets หรือ GitLab Variables

ความเสี่ยงระดับวิกฤต:

- หากแพ็คเกจ npm ตัวใดตัวหนึ่งแฝงมัลแวร์ใน Build Runner ผู้โจมตีจะขโมยกุญแจคลัสเตอร์ไปได้ทันที
- ไฟร์วอลล์ขององค์กรต้องยอมเจาะพอร์ตขาเข้า เพื่อให้เซิร์ฟเวอร์ CI ภายนอกยังเข้าสู่ Kubernetes API
- การแก้ไขสัดหน้างานบนคลัสเตอร์ (`kubectl edit`) จะไม่ถูกบันทึกกลับเข้ามาใน Git เลย

การปฏิบัติด้วยระบบ Pull-Model (GitOps)

เอเจนต์ GitOps (Argo CD) จะทำงานอยู่ **ภายใน** คลัสเตอร์ Kubernetes ที่ได้รับการคุ้มกันอย่างมิดชิด

- ไม่มีการฝาก Credential คลัสเตอร์ไว้ในเครื่องมือ CI ภายนอกเลยแม้แต่ชิ้นเดียว
- ใช้การต่อออกภายนอกผ่าน HTTPS แบบ Read-Only สู่ Git ไม่ต้องเปิดพอร์ตขาเข้าบนไฟร์วอลล์
- ระบบกระหนบยอดสถานะต่อเนื่องจะย้อนการแก้ไขสัดที่แอบทำหน้างานกลับสู่ Git อัตโนมัติ

สี่หลักการ OpenGitOps ตามมาตรฐาน CNCF

ข้อกำหนดอย่างเป็นทางการที่กำกับการประสานและกระทบยอดสถานะระบบแบบ Declarative

Declarative

1

สถานะเป้าหมายของระบบถูกประกาศอย่างชัดเจนในรูปแบบ Kubernetes YAML / Helm / Kustomize

Versioned

2

สถานะเป้าหมายถูกจัดเก็บบน Git พร้อมประวัติการแก้ไข และเลข Commit SHA ที่เปลี่ยนแปลงไม่ได้

Pulled Auto

3

เอเจนต์ภายในคลัสเตอร์ดึงสถานะมาปรับใช้อัตโนมัติ โดยไม่มีคำสั่ง Push จากภายนอก

Reconciled

4

เฟาส์เกตสถานะจริงบนคลัสเตอร์อย่างต่อเนื่อง และกู้คืนการเบี่ยงเบนกลับสู่ Git อัตโนมัติ

"Git คือแหล่งความจริงเดียว (Single Source of Truth) สำหรับโครงสร้างพื้นฐานและสถานะแอปพลิเคชัน สิ่งใดที่อยู่ใน Git ย่อมปรากฏบนคลัสเตอร์ สิ่งใดที่ไม่อยู่ใน Git ย่อมไม่มีตัวตนอยู่จริง"

การแยก Repository ระหว่างโค้ดและคอนฟิ็ก และกลไก In-Cluster Pull

จัดโครงสร้างแยกคลังซอร์สโค้ดของแอปพลิเคชัน ออกจากคลังคอนฟิ็ก Declarative GitOps อย่างเป็นระเบียบ

การแยกคลังโค้ดอย่างชัดเจน (Repository Separation)

```
REPO 1: app-source-code.git (สำหรับนักพัฒนา)
├─ src/
├─ package.json
└─ Dockerfile
# ทริกเกอร์ CI: รันเทสต์, บิลด์อิมเมจ, พูช Digest

REPO 2: workshop-gitops.git (สำหรับ Argo CD)
├─ base/
│   ├── deployment.yaml
│   └─ service.yaml
└─ overlays/
    ├── staging/
    │   └─ kustomization.yaml (image: v1.3.0-rc1)
    └─ production/
        └─ kustomization.yaml (image: v1.2.0)
# Argo CD จะเฝ้ามอง Repo นี้และซิงก์เข้าสู่คลัสเตอร์
```

ทำไมการแยก Repo จึงมีความสำคัญอย่างยิ่ง

- **ป้องกัน Infinite Loop:** หาก CI แก้ไข Manifest ใน Repo เดียวกัน จะทำให้เกิดการทริกเกอร์บิลด์ CI วนลูบไม่รู้จบ
- **หลักการ Least Privilege:** นักพัฒนามีสักสิทธิ์คอมมิตเฉพาะในโค้ด มีเพียงบอตออกรุ่นและทีมแพลตฟอร์มเท่านั้นที่แตะต้องคอนฟิ็กโปรดักชันได้
- **การเลือกระดับหลายสภาพแวดล้อม:** เลื่อนเวอร์ชันของอิมเมจข้าม Staging และ Production ได้ง่ายๆ เพียงเปลี่ยนแท็กใน Kustomize

พิมพ์เขียวสถาปัตยกรรมความปลอดภัยตามแนวทาง GitOps

หลักเกณฑ์พื้นฐานในการดูแลคลัสเตอร์ Kubernetes แบบ Zero-Trust ด้วยระบบ GitOps

Pull-Only

ปิดพอร์ตขาเข้า

ปิดพอร์ตบริหารจัดการทั้งหมดบนไฟร์วอลล์ เอเจนต์ดึงข้อมูลผ่าน Egress HTTPS เท่านั้น

ไร้การเบี่ยงเบน

กระตบยอดอัตโนมัติ

Argo CD ขจัดปัญหาการตั้งค่าเพี้ยน ด้วยการบังคับใช้สถานะเป้าหมายจาก Git อยู่ตลอดเวลา

Audit Log

ประวัติคอมมิต

ทุกการเปลี่ยนแปลงมีชื่อผู้ทำ, เวลาที่แนชด และหลักฐานการรีวิวที่แก้ไขไม่ได้

ย้อนคืนฉับไว

MTTR < 2 นาที

ย้อนคืนระบบโปรดักชันได้ทันที เพียงสั่ง `git revert` ครั้งเดียวใน Config Repo

สรุป GitOps: การยึด Git เป็นแหล่งความจริงเดียว จะช่วยขจัดความเสี่ยงจากการเปิดเผย Credential คลัสเตอร์ และเปลี่ยนการดีพลอยให้เป็นการประสานข้อมูลที่แม่นยำและแน่นอน

อันตรายจาก Configuration Drift และการแก้สดหน้างาน

เมื่อการแก้ปัญหาเฉพาะหน้าด้วยมือสร้างคลัสเตอร์แบบ Snowflake ที่ไม่สามารถกู้คืนได้เมื่อเกิดวิกฤต

กลุ่มอาการคลัสเตอร์แบบ Snowflake (Snowflake Syndrome)

ระหว่างที่ระบบกำลังล่ม วิศวกรเปิดเทอร์มินัลแล้วสั่ง:

```
kubectl edit deployment/order-service
# แก้ไข memory limit จาก 512Mi เป็น 2Gi ด้วยมือทันที
```

ปัญหาเฉพาะหน้าคลี่คลายลงได้ แต่การเปลี่ยนแปลงนี้ไม่เคยถูกส่งกลับเข้า Git เลย

หกเดือนต่อมา เมื่อคลัสเตอร์พัง: เมื่อทีมสร้างคลัสเตอร์ใหม่ขึ้นมา ค่าเมโมรี 2Gi ที่เคยแก้ไขหายไ้ไป ระบบจึงแครชซ้ำ ซ้อนภายใต้กราฟฟิคปกติ โดยไม่มีใครจำได้เลยว่าเคยแก้สดอะไรไว้บ้าง

การกู้คืนตัวเองอัตโนมัติของ Argo CD (Self-Healing)

Argo CD จะตรวจสอบเปรียบเทียบสถานะจริงบนคลัสเตอร์กับ Git **ทุกๆ 3 นาที** (หรือทันทีผ่าน Git Webhooks)

หากใครบังอาจแก้ไขคลัสเตอร์ด้วยมือ Argo CD จะตรวจพบความเพี้ยน (OutOfSync) และเขียนทับกลับสู่สิ่งที่ระบุใน Git ทันทีโดยอัตโนมัติ

ในบทเรียนนี้: เราจะดีพลอยระบบจริง, จัดลำดับการปล่อยด้วย Sync Waves และทดลองແຄລ้งแก้ไขคลัสเตอร์เพื่อดูระบบกู้คืนตัวเอง

สถาปัตยกรรมคอนโทรลเลอร์ของ Argo CD

โครงสร้างองค์ประกอบภายในที่ขับเคลื่อนการทยอยสถานะ: GitOps ในระดับองค์กร

01

API Server

ให้บริการ gRPC/REST APIs และหน้าจอ Web UI ดูแลเรื่องการยืนยันตัวตน, สิทธิ์ RBAC, ระบบ SSO (OIDC/Dex) และสตรีม Audit Log

02

Repository Server

ทำหน้าที่เก็บแคชของ Git Repositories ในเครื่อง แปลง Kustomize Overlays และ Helm Templates ให้ออกมาเป็น Kubernetes Manifests

03

Application Controller

หัวใจหลักของระบบทยอยสถานะ: คอยเปรียบเทียบคลัสเตอร์จริงกับ Git และสั่งทำ Self-Healing เพื่อแก้ไขความคลาดเคลื่อน

หัวใจของ Application CRD: ทำหน้าที่ผูกเส้นทางโฟลเดอร์ใน Git (Source) เข้ากับ Namespace บน Kubernetes (Destination) พร้อมกำหนดนโยบาย Sync, Prune และ Self-Heal

ซิงก์หลายระดับด้วย Sync Waves และการกู้คืนตัวเองจาก Chaos

ดีพลอยเวิร์กโหลดที่มีการจัดลำดับชั้น และพิสูจน์การกู้คืนสถานะอัตโนมัติเมื่อเกิดการแก้ไขสด

04-gitops-manifests/sync-waves

```
# Wave 0: งาน Database Migration (รันก่อน)
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: Sync
    argocd.argoproj.io/sync-wave: "0"
---
# Wave 1: Backend Deployment (รันทีหลัง)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  annotations:
    argocd.argoproj.io/sync-wave: "1"
# จะเริ่มดีพลอยเมื่อ Wave 0 รันผ่านสำเร็จแล้วเท่านั้น!
```

จำลองการสร้างความโกลาหล (Drift Simulation)

```
cd workshop-setup/06-chaos-and-verification

# macOS & Linux:
chmod +x simulate-drift.sh && ./simulate-drift.sh

# Windows (PowerShell):
.\simulate-drift.ps1

# 1. แอบสั่งลด Replicas เป็น 1 บนคลัสเตอร์ด้วยมือ:
kubectl scale deployment/order-service --replicas=1

# 2. Argo CD ตรวจพบสถานะ OutOfSync
# 3. กลไก Self-Healing สั่งดีด Replicas กลับมาเป็น 3 ทันที!
SUCCESS: ตรวจพบความเพี้ยนและกู้คืนกลับสู่สถานะเดิมอัตโนมัติ!
```

ไ้การแทรกแซงด้วยมือ: การตั้งค่าบนโปรดักชันถูกล็อกไว้กับ Git การแก้ไขที่แปลกปลอมจะถูกกลบล้างในเวลาไม่กี่วินาที

คู่มือการใช้งาน Argo CD Runbook และตารางสถานะสุขภาพ

การวินิจฉัยและจัดการเวิร์กโหลด GitOps ในสภาพแวดล้อมโปรดักชันระดับองค์กร

Synced

สถานะปกติ

สถานะจริงบนคลัสเตอร์ตรงกับสิ่งที่ระบุไว้ใน Git อย่างสมบูรณ์แบบ

OutOfSync

ตรวจพบความเพี้ยน

มีคอมมิตใหม่บน Git หรือมีการแก้สดหน้างาน; ระบบกำลัง
สั่งกู้คืน

Degraded

POD มีปัญหา

Pod ไม่ผ่านการตรวจสอบสุขภาพ; ต้องใช้ `kubectl describe` ตรวจสอบดูลือก

Sync Waves

จัดลำดับการปล่อย

จัดลำดับการทำงานของ Migration, Secrets, และ Services อย่างแม่นยำ

สรุป Argo CD: Argo CD ช่วยเปลี่ยนผ่าน Kubernetes จากคลัสเตอร์ที่ต้องคอยดูแลด้วยมือ ให้กลายเป็นแพลตฟอร์มอัตโนมัติที่มีระบบรักษาตัวเองในตัว

ภัยจากการเผลอใส่ Secret เป็น Plaintext ลงใน Git

ข้อผิดพลาดร้ายแรงอันดับ 1 ด้านความปลอดภัยในการก้าวสู่ระบบ GitOps ขององค์กร

ความเข้าใจผิดมหันต์เรื่อง Base64 Secret

นักพัฒนาได้ยืมว่าแนวทาง GitOps ต้องนำ *ทุกอย่าง* เข้าสู่ Git จึงสร้าง Kubernetes Secret แบบนี้:

```
apiVersion: v1
kind: Secret
data:
  password: c3VwZXItc2VjcmV0LXBhc3N3b3Jk # base64!
```

ความจริงอันน่าสะพรึงกลัว: Base64 เป็นเพียงการเข้ารหัสเพื่อการส่งต่อ (Encoding) ไม่ใช่การเข้ารหัสลับ (Encryption) ใครก็ตามที่มองเห็น Git Repo จะสามารถถอดรหัสผ่านได้ใน 1 มิลิวินาทีด้วย `echo ... | base64 -d`

กฎเหล็ก: ห้ามเก็บรหัสผ่าน Plaintext ใน Git

ห้ามคอมมิตรหัสผ่าน, API Token หรือ Private Key ที่ยังไม่ได้เข้ารหัสลงใน Git อย่างเด็ดขาด แม้จะเป็น Private Repository ก็ตาม

ทางออกระดับองค์กร: ใช้ **External Secrets Operator (ESO)** เพื่อแยกการประกาศคอนฟิกของ Secret ออกจากค่าความลับที่แท้จริง

External Secrets Operator (ESO) และการกู้คืนระบบใน 15 นาที

ดึงข้อมูลความลับจาก Enterprise Vault และการบรรจุเป้าหมายกู้คืนคลัสเตอร์ทั้งระบบภายใน 15 นาที

1. Enterprise Vault / Cloud KMS

เก็บรหัสผ่านหลักไว้ใน HashiCorp Vault หรือ PROEN Cloud KMS

↓ ดึงข้อมูลผ่านสิทธิ์ที่ปลอดภัย

2. External Secrets Operator (ในคลัสเตอร์)

อ่านไฟล์ ExternalSecret Declarative จาก Git

↓ ผลิตเป็น Secret ในแบบ tmpfs ของ K8s

3. Kubernetes Native Secret

นำไป Mount เข้าเป็น Environment Variables ใน Pod ของแอปพลิเคชัน

ลำดับขั้นตอนการกู้คืนระบบใน 15 นาที (DR Sequence)

เนื่องจากสถานะเป้าหมายทั้งหมดถูกบันทึกไว้อย่างสมบูรณ์ใน Git:

- **T+0:** ประกาศภาวะฉุกเฉิน ดาต้าเซ็นเตอร์เสียหายทั้งหมด
- **T+3m:** สร้างคลัสเตอร์ Kubernetes ใหม่บน PROEN Cloud
- **T+5m:** ติดตั้ง Argo CD Operator ด้วยคำสั่ง Helm ธรรมดาเดียว
- **T+7m:** สั่งรัน Manifest รุกของ "App-of-Apps"
- **T+15m:** ไมโครเซอร์วิสทั้ง 50 ตัวฟื้นคืนชีพกลับมาสมบูรณ์แบบจาก Git!

สร้าง ExternalSecret Manifest และจำลองการกู้คืนคลัสเตอร์ใหม่

ประกาศ ExternalSecret ที่เชื่อมโยงสู่ Secure Vault และซ้อมรับมือการกู้คืนระบบเมื่อเกิดภัยพิบัติ

ข้อกำหนด ExternalSecret (บันทึกลงใน Git)

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: order-service-secrets
  namespace: production
spec:
  refreshInterval: 1h # หมุนเวียนคีย์อัตโนมัติทุก 1 ชม.
  secretStoreRef:
    name: vault-backend
    kind: ClusterSecretStore
  target:
    name: db-credentials # ชื่อ Secret ใน K8s
  data:
    - secretKey: database-url
      remoteRef:
        key: production/gvents/db
        property: connection_string
```

ขั้นตอนการซ้อมกู้คืนระบบฉุกเฉิน (DR Drill)

```
# 1. โคลนคลังคอนฟิก GitOps
git clone https://github.com/3lVv0w/code-to-cloud-workshop.git

# 2. ติดตั้ง Argo CD บนคลัสเตอร์ใหม่
cd workshop-setup/05-argocd
# macOS & Linux:
chmod +x install-argocd.sh && ./install-argocd.sh
# Windows (PowerShell):
.\install-argocd.ps1

# 3. นำ Root Application ขึ้นระบบ (ทุก OS)
kubectl apply -f application.yaml

# 4. สังเกตการณ์ทุก Namespace, Pods และ Config
# กระทบยอดและฟื้นตัวกลับมาอย่างสมบูรณ์ในไม่กี่นาที!
```

แผนกลยุทธ์ 90 วันสู่ Cloud-Native ในระดับองค์กร

แผนการเปลี่ยนผ่านอย่างเป็นขั้นตอน เพื่อย้ายเวิร์กโหลดระดับองค์กรสู่ PROEN Cloud ด้วยระบบ GitOps

เดือนที่ 1

รากฐานและการเพิ่มความแข็งแกร่ง

- ปรับปรุงเซอร์วิสตามหลัก 12-Factor Disposability
- เปลี่ยนผ่านสู่ Google Distroless Base Images
- ติดตั้งระบบอัตโนมัติเพื่อสร้าง SBOM และเซ็นอิมเมจด้วย Cosign ใน CI

เดือนที่ 2

แพลตฟอร์มและเอนจิน GitOps

- สร้างคลัสเตอร์ Kubernetes บน PROEN Cloud
- ติดตั้ง Argo CD และแยกคลัง Code กับ Config Repos ออกจากกัน
- เชื่อมต่อ External Secrets Operator เข้ากับระบบ Enterprise Vault

เดือนที่ 3

การปล่อยโปรดักชันและซ้อม DR

- เริ่มใช้งาน Canary Progressive Delivery ผ่าน Argo Rollouts
- ติดตั้งตัวชี้วัด OpenTelemetry และ Structured Logging
- ทดสอบซ้อมแผนกู้คืนระบบฉุกเฉิน (DR Drill) ให้สำเร็จภายใน 15 นาที

กฎเหล็กของแผนงาน: ก้าวไปข้างหน้าอย่างเป็นขั้นตอน อย่าพยายามทำแบบ "Big-Bang" ครั้งเดียว แต่ให้สร้างความมั่นใจในสภาพแวดล้อม Staging ผ่าน GitOps ก่อนที่จะย้ายเวิร์กโหลดจริงบนโปรดักชัน

ยกระดับวิศวกรรมซอฟต์แวร์สู่ **ความเร็วระดับ Cloud-Native**

การประกาศสถานะแบบ Declarative • ห่วงโซ่อุปกรณ์ที่ตรวจสอบได้ • รับมือ Concurrency อย่างแข็งแกร่ง • ระบบ GitOps แบบ Zero-Trust

คลังโค้ดแล็บอย่างเป็นทางการ

`https://github.com/31Vv0w/code-to-cloud-workshop.git`

คณะทำงาน OpenGitOps สากา

`https://opengitops.dev`

สมาคมโปรแกรมเมอร์ไทย

`https://thaiprogrammer.org`

คุณสุภรรัตน์ จิรกุลพรชัย (CTO, Jumpbox)

คุณกฤษฎา วิวอก (กรรมการ สมาคมโปรแกรมเมอร์ไทย)

คุณชาลี คัมภีรภาพ (Solution Architect, VMware by Broadcom)